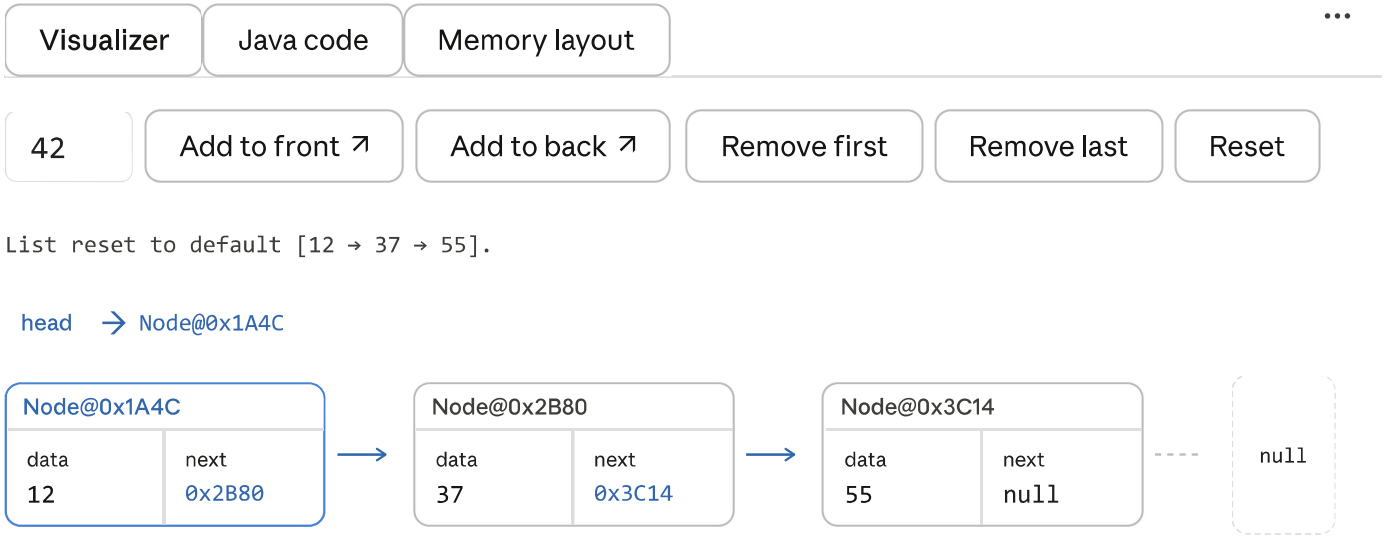
Linked list data structure visualizer

I want to create a visualizer for how data is stored in a linked list using objected-oriented programming in Java.



Here's an interactive linked list visualizer with three views:

Visualizer tab — shows the live chain of `Node` objects. Each box displays the `data` field and the `next` pointer (the address of the following node). The blue-bordered node is always the `head`. Try adding and removing nodes to watch how the pointers update.

Java code tab — the two core classes: `Node` (holds `data` and `next`) and `LinkedList` (manages the chain via a `head` reference). Key methods like `addFirst`, `addLast`, and `removeFirst` are annotated with their time complexity.

Memory layout tab — shows each node as it exists on the heap: a simulated address, 4 bytes for the `int data`, and 8 bytes for the `next` reference. This makes it clear that nodes aren't stored contiguously in memory — they're linked purely through references.

The fundamental idea is that a linked list is just a chain of `Node` objects where each one knows the address of its neighbor. The list itself (`LinkedList`) only needs to remember where the chain *starts* — everything else is reachable by following `next` pointers.

i want a java code for it

Ran a command, created a file, read a file >

The file includes everything in a single `LinkedList.java`:

`Node` **(inner class)** — holds `data` (int) and `next` (reference to the next node).

`LinkedList` — manages the chain with these methods:

Method	Description	Time
`addFirst(data)`	Insert at head	O(1)
`addLast(data)`	Insert at tail	O(n)
`addAt(index, data)`	Insert at position	O(n)
`removeFirst()`	Remove head	O(1)
`removeLast()`	Remove tail	O(n)
`remove(data)`	Remove by value	O(n)
`contains(data)`	Search	O(n)
`get(index)`	Access by index	O(n)
`reverse()`	Reverse in place	O(n)
`print()`	Print the chain	O(n)

To run it, compile and execute with:

```bash
javac LinkedList.java
java LinkedList
```

The `main` method demos all operations with expected output shown in comments.

Linkedlist
Code · JAVA

Download

put the main method in a different file

Ran a command, edited a file, created a file >

Done. The `main` method is now in `Main.java` and `LinkedList.java` is clean with just the data structure. To compile and run both together:

```bash
javac LinkedList.java Main.java
java Main
```

Linkedlist
Code · JAVA
Download

Main
Code · JAVA
Download

Download all

I want to allow the user to manipulate the linked list through a UI. Help me build that UI in a separate class

Check existing files >

I'll build a Swing-based GUI class (`LinkedListUI.java`) that lets the user interact with your `LinkedList` visually, and update `Main.java` to launch it.

Swing UI class for interacting with the LinkedList >

Now update `Main.java` to launch the UI instead of the console demo:

Ran a command, read a file >

Here's what was built. The project is now three files:

- `LinkedList.java` — unchanged, pure data structure

- `LinkedListUI.java` — Swing window with a sidebar and canvas
- `Main.java` — launches the UI on the Swing event thread

What the UI includes:

- **Left sidebar** — grouped buttons for every operation (add front/back/at-index, remove first/last/by-value, search, get by index, reverse, clear), with separate value and index input fields
- **Canvas** — draws each node as a split box showing `data` and `next` fields, with arrows between them and a `null` sentinel at the tail. The `HEAD` node has a blue border. Searched/retrieved nodes flash amber for 1.5 seconds.
- **Status bar** — shows the last operation and its result in green (success) or red (error)

To compile and run:

```bash
javac LinkedList.java LinkedListUI.java Main.java
java Main
```

Linkedlistui
Code · JAVA
Download

Main
Code · JAVA
Download

Linkedlist
Code · JAVA
Download

Download all

This is good, I also want the user to be able to iterate through the list. I also don't want to use a custom implementation of linked list

Two changes needed: swap the custom `LinkedList` for `java.util.LinkedList`, and add iteration controls to the UI. Let me update both files.

Review current LinkedListUI.java >

Now I have a clear picture of the full file. I'll rewrite `LinkedListUI.java` to use `java.util.LinkedList` and add iterator controls, and delete the old `LinkedList.java`.

Ran 3 commands >

Now update `Main.java` — it no longer needs to import anything since `LinkedListUI` handles everything:

Ran a command, read a file >

Two key changes made:

Custom `LinkedList.java` is gone. `LinkedListUI` now imports and uses `java.util.LinkedList<Integer>` directly, so all operations (`addFirst`, `addLast`, `remove`, `contains`, `get`, etc.) are the standard library methods. `Collections.reverse()` replaces the custom reverse logic.

New Iterator section in the sidebar — backed by a real `ListIterator<Integer>` from `list.listIterator()`:

Button	Method called	What it does
Start iterator	`list.listIterator(0)`	Places cursor before index 0
→ Next	`iterator.next()`	Advances cursor, highlights that node in green
← Previous	`iterator.previous()`	Moves cursor back
✎ Set value	`iterator.set(v)`	Replaces the last visited node's value
✕ Remove current	`iterator.remove()`	Removes the last visited node
■ End iterator	—	Clears the iterator and resets the canvas

The green **CURSOR** badge and triangle indicator on the canvas show exactly which node the iterator is currently positioned on. Iterator buttons are automatically enabled/disabled based on state — for example, `next` grays out when `hasNext()` is false, and `set` / `remove` stay disabled until `next()` or `previous()` has been called.

Any structural modification (add, remove, clear, reverse) from the sidebar resets the iterator, mirroring how `ConcurrentModificationException` works in real Java code.

To compile and run (no `LinkedList.java` needed anymore):

```bash
javac LinkedListUI.java Main.java
java Main
```

Linkedlistui
Code · JAVA
Download

Main
Code · JAVA
Download

Download all

can we reorganize the code, the LinkedListUI should only handle displaying, the code that makes the buttons run should be in another file (maybe call it LinkedListController or something)

Good separation of concerns. I'll create `LinkedListController.java` to own all the action logic, and trim `LinkedListUI` down to pure display + wiring.

Read full LinkedListUI.java to plan the split

Clear picture. The plan:

- **`LinkedListController`** — owns the `LinkedList`, `ListIterator`, all `do*` methods, input parsing, and iterator button state
- **`LinkedListUI`** — owns the window, layout, canvas drawing, and wires buttons to the controller
- The controller calls back into the UI via a simple interface to update status and canvas

Created a file, ran a command, read a file

Here's how the responsibilities are now divided:

`LinkedListController` owns everything that *does* something — the `LinkedList`, the `ListIterator`, all mutating and querying methods. It never touches Swing. When an operation completes, it tells the UI what happened by calling methods on the `Callback` interface: `onStatusOk`, `onListChanged`, `onHighlight`, `onIteratorMoved`, etc.

`LinkedListUI` implements that `Callback` interface and owns everything visual — building panels, drawing the canvas, styling buttons. Its button lambdas are now one-liners that either call the controller directly (`e -> controller.reverse()`) or parse a field first and forward (`e -> withValue(controller::addFirst)`). It has zero list logic of its own.

The `Callback` interface is the clean boundary between the two — the controller never imports Swing, and the UI never touches the `LinkedList` directly.

bash

```
javac LinkedListController.java LinkedListUI.java Main.java
java Main
```

Linkedlistcontroller
Code · JAVA
Download

Linkedlistui
Code · JAVA
Download

Main
Code · JAVA
Download

Download all

> LinkedListUI.java line 61 (controller.onIteratorButtonsChanged(false, false, false, false);) error, can't resolve method onIteratorButtonsChanged in file LinkedListController

The method `onIteratorButtonsChanged` is on the `Callback` interface, not on the controller. That call should go through `this` (since `LinkedListUI` implements `Callback`), or better yet, just let the controller's constructor do it via `notifyIterButtons`.

Edited 2 files, read a file ›

`onIteratorButtonsChanged` is a method on the `Callback` interface, which `LinkedListUI` implements — so the call belongs on `this`, not on the controller. Changed

`controller.onIteratorButtonsChanged(...)` to just `onIteratorButtonsChanged(...)`.